

Paafekt FurnitureFit — Patent vs Codebase Audit

Technical cross-check of six provisional patent applications against the iOS (Swift) and Android (Kotlin) implementation

Patent Technical Audit

April 2026

Contents

0. Scope, Method, and Disclaimers	2
1. Codebase Map (Working Reference)	3
1.1 Directories	3
1.2 Models Actually Bundled	4
1.3 Top Files by Audit Relevance	4
2. Per-Patent Audit (Bullet Inaccuracies + Verdict)	5
Patent 1 — On-Device 3D Room Reconstruction Without Depth Sensors	5
Patent 2 — Neural Object Segmentation for Architectural Surface Identification	7
Patent 3 — Centimetre-Accurate Spatial Measurement	8
Patent 4 — AR Furniture Placement and Visualization Pipeline	9
Patent 5 — On-Device Neural Architecture for Resource-Constrained Hardware	10
Patent 6 — Integrated End-to-End Furniture Visualization System	12
3. Gap Report — Methods in Code Not Described in Any Patent	13
GAP-1 — Single-image neural Gaussian-splat reconstruction as a primary embodiment . . .	13
GAP-2 — Hybrid metric-scale estimation: AR-tracking-derived anchors fused with neural monocular depth via coefficient-of-variation gating	14
GAP-3 — User-supplied two-point calibration for monocular metric scale recovery	15
GAP-4 — EXIF-focal-length-conditioned dimension extraction with shot-type branching . . .	15
GAP-5 — Memory-budget-conditioned model loading with explicit refusal	16
GAP-6 — Dual neural inference backends behind a single capability-detected facade	16
GAP-7 — Manual room boundary capture as a non-ML fallback to neural reconstruction . . .	17
GAP-8 — Camera contention detection and dual-camera arbitration	18
GAP-9 — Quantile-trimmed depth-grid AABB room measurement	18
GAP-10 — Bundle-id-and-on-demand-resource hybrid model delivery	19
4. Surgical Claim Fixes (Quote Old → Suggest New)	19
Patent 1	19
Patent 2	21
Patent 3	22
Patent 4	22
Patent 5	23
Patent 6	24
5. Loophole Check — Competitor Workarounds and How to Close Them	24
Patent 1	24

Patent 2	24
Patent 3	25
Patent 4	25
Patent 5	26
Patent 6	26
6. Cross-Patent Consistency Notes	26
7. Filing-Day Triage Checklist	27

0. Scope, Method, and Disclaimers

What this document is. A technical, file-grounded audit comparing the six provisional patent applications in `Untitled_PAAFEKT_INC_USPTO_PROVISIONAL_PATENT_Bur_kS4r.txt` against the actual code at `/Users/al/Documents/tries01/Furnit/`. It produces:

- **§2** — Per-patent verdict with bulleted inaccuracies (patent says X, code does Y)
- **§3** — Gap Report: methods present in code that are not described in any patent, each with insertable 2–3 paragraph prose
- **§4** — Surgical Claim Fixes: quoted current claim language → suggested replacement
- **§5** — Loophole Check: one paragraph per vulnerability with a suggested fix
- **§6** — Cross-patent consistency notes

What this document is NOT. It is not legal advice. It does not address USPTO filing mechanics (ADS, SB/15A micro-entity certification, Patent Center upload, fees, drawings). It does not evaluate prior-art-based novelty or non-obviousness. A registered patent attorney must review before filing if any meaningful change to claim scope is intended.

Source cited convention. File paths are absolute (`/Users/al/Documents/tries01/Furnit/...`). Line citations use `file:line` format. Where something was searched for and not found, the entry is marked **NOT FOUND** rather than guessed.

Branding. “Paafekt” (patents) and “FurnitureFit” / “Furnit” (codebase) refer to the same product.

1. Codebase Map (Working Reference)

1.1 Directories

Path	What it is
Furnit/	iOS app target (Swift)
FurnitTests/	iOS unit tests
android/app/src/main/java/ com/furnit/android/	Android app source (Kotlin)
android/app/src/main/assets/	Android bundled ML models (ONNX, NCNN)
android/models_cpu/, android/ models_cpuvulkan_hybrid/, android/sharp_vulkan_only/, android/executorch_models/ SHARP_fp32_1536.mlpackage/	ExecuTorch .pte SHARP pipeline (~18 GB combined across flavours)
yoloe-11l-seg-pf.mlpackage/, exports/yoloe-26l-seg- pf.mlpackage/	iOS SHARP Core ML model (~1.2 GB) iOS YOLO-E segmentation Core ML (~60–61 MB)
beeware/src/furnit_beeware/	BeeWare/Toga Python prototype (mostly placeholders + viewer)
scripts/, android/pyfiles/, android/scripts/	Model conversion, export, and inspection scripts
docs/, android/docs/	Architecture and ops docs (gold for cross-checking against patents)

1.2 Models Actually Bundled

Model	Format	Precision	Where Loaded	Compute
SHARP_fp32_16Core ML .mlpackage (iOS)	FP32 (graph contains FP16 casts)	Furnit/Services/OnDevice/SHARPService.swift	computeUnits = .cpuOnly	
SHARP (split, multi-part)	Execu-Torch .pte (Android)	INT8 (CPU) and FP32 (Vulkan) flavours	services/ExecutorchInt8Sharp.kt	CPU XN-NNPACK or Vulkan delegate
yolo-e-11l-seg-pf	Core ML .mlpackage (iOS)	FP32 weights / FP16 outputs	Services/OnDevice/YOLOEModelService.swift	computeUnits = .cpuOnly (per code comments, due to SIGABRT on GPU/ANE)
yolo-e-11l-seg-pf.onnx	ONNX (Android)	FP32	services/FurnitureFitManager.kt	DefaultSessionOptions() (CPU; no NNAPI / no XNNPACK delegate enabled)
yolo-e-11l-seg.{param,bin}, yolo-e-pf.{param,bin}	NCNN (Android)	FP16/FP32 binary	services/NcnnYolo-e.kt, YoloEImageInference.kt	NCNN CPU

1.3 Top Files by Audit Relevance

iOS (Swift)

1. Furnit/Services/OnDevice/SHARPService.swift — Core ML SHARP inference, PLY export, on-demand-resource loading, memory gates
2. Furnit/Views/FurnitureFit/FurnitureFitView.swift — Live YOLO-E segmentation, AR/AV-Capture hybrid, overlay scaling
3. Furnit/Services/OnDevice/YOLOEModelService.swift — YOLO-E Core ML loader (cpuOnly)
4. Furnit/Views/FurnitureFit/FurnitureFitARSupport.swift — ARKit world tracking config, scene-depth handling, percentile depth sampling, distance gate $0.1 < d < 50$ m
5. Furnit/Views/Components/GaussianSplatView.swift — Metal splat renderer, depth read-back, room measurement, furniture placement
6. Furnit/Models/RoomGeometryEngine.swift — Point cloud → planes (RANSAC) → RoomModel; ceiling height constant 2.44 m
7. Furnit/Services/SplatDepthQuery.swift — TwoPointSplatCalibration (user-supplied known distance)
8. Furnit/Services/RoomReconstruction/SinglePhotoRoomReconstructor.swift — **Legacy** SceneKit five-plane room from photo, door height = 2.1 m, person fallback 1.7 m

9. `Furnit/Services/RoomReconstruction/MiDaSDepthEstimator.swift` — **Stub**
(`generateSyntheticDepthMap`)
10. `Furnit/AR/RealityKitObjectPlacementManager.swift` — `arView.raycast` placement, distance clamp 1–8 m
11. `Furnit/Models/QualitySettings.swift` — LiDAR support detection, render quality enum

Android (Kotlin)

1. `services/ExecutorchInt8Sharp.kt` — ExecuTorch SHARP pipeline, JNI monodepth (`getLastMonodepthInfoNative`), PLY write
2. `services/SharpService.kt` — Orchestration; async `room_dims_v7` measurement
3. `utils/SharpRoomDimensionsV7.kt` — PLY-based room dimension extraction with optional EXIF focal branching (STRAIGHT / CORNER / FALLBACK_NO_FOCAL)
4. `ar/FurnitureFitArMetrics.kt` — ARCore Depth16 sampling, median + IQR, plane raycast fallback
5. `ar/FurnitureFitArCameraController.kt` — ARCore Session, `Config.DepthMode.AUTOMATIC`, frame throttling (`minFrameIntervalMs = 55L`)
6. `services/FurnitureFitManager.kt` — ONNX YOLO-E inference (default CPU)
7. `services/MetricScaleEstimator.kt` — Fuses ARCore-anchor distances with monodepth ratios; **gates on coefficient-of-variation ≤ 0.5**
8. `FurnitureFitOverlayView.kt` — 2D overlay (drag/pinch); 2D bitmap mask hit-test
9. `services/SinglePhotoRoomReconstructor.kt` — Manual boundaries → 5-plane GLB (parity with iOS legacy)
10. `RoomBoundaryActivity.kt` — **Manual** wall/floor draggable lines (non-NN)

2. Per-Patent Audit (Bullet Inaccuracies + Verdict)

Patent 1 — On-Device 3D Room Reconstruction Without Depth Sensors

Verdict: NEEDS REVISION (significant)

The patent describes a multi-view photogrammetry pipeline (feature extraction → matching → five-point pose → bundle adjustment → per-frame neural depth → multi-view voxel fusion → mesh extraction → scale resolver). The actual production reconstruction is a **single-image 3D Gaussian Splatting model (SHARP)** producing a PLY point set, with VIO/world-tracking handled by the platform AR frameworks (ARKit / ARCore). Several claims describe an entire pipeline that does not exist in the codebase.

Inaccuracies (file:line cited)

- **Claim 1 step “extract visual features ... estimate camera poses ... feature correspondences” + DD §3, §4 (“five-point algorithm or equivalent minimal solver”, “bundle adjustment ... sparse nonlinear least-squares”): NOT FOUND.** A repo-wide `grep` for `bundleAdjust|BundleAdjust|fivepoint|fivePoint|five_point|essentialMatrix|fundamentalMatrix|featureMatching` returns **zero hits in .swift and .kt**. The “feature extraction → matching → BA” pipeline simply does not exist in code. Where SLAM/VIO is needed, the code delegates to ARKit (`ARWorldTrackingConfiguration` in `FurnitureFitARSupport.swift:87–96`) or ARCore (`Session/Config` in `FurnitureFitArCameraController.kt:454–461`).

- **Claim 8 “quantized neural network keypoint detector and descriptor on the mobile computing device’s neural processing hardware”: NOT FOUND.** No neural keypoint detector/descriptor model is bundled or invoked.
- **Claim 9–10 “convolutional neural network ... per-image depth map ... quantized for execution on a neural processing unit”:**
 - iOS MiDaSDepthEstimator.swift:14–35 is a stub: `if model != nil { ... } logDebug("Using synthetic depth map (fallback)"); return generateSyntheticDepthMap(from: ciImage)`. No real depth CNN is loaded.
 - The actual deployed neural model is **SHARP (single-image 3D Gaussian Splatter)** — not a per-frame dense depth network. SHARP produces Gaussian primitives as named output tensors `var_5420 / var_5424 / var_5412 / etc.` (SHARPService.swift:842–866).
 - Where dense monodepth does exist, it’s a *byproduct* inside the Android ExecuTorch SHARP native pipeline, exposed via `getLastMonodepthInfoNative()` and serialised to `sharp_monodepth.bin` as `Float32` (ExecutorchInt8Sharp.kt:974–999).
 - **Quantization to NPU: FALSE for the production iOS path.** Both YOLO-E and SHARP are loaded with `MLModelConfiguration.computeUnits = .cpuOnly` (SHARPService.swift:411–416; YOLOEModelService.swift ~205–209). Code comments cite stability (SIGABRT on GPU/ANE). Nothing in the iOS production path executes neural inference on the NPU.
- **DD §6 “multi-view depth fusion ... voxel grid ... back-projecting each per-frame depth map ... weighted average ... robust statistical estimation”: NOT FOUND.** Production reconstruction is **batch single-image SHARP** → **PLY**. There is no voxel grid, no per-frame depth fusion, no weighted accumulation across multiple views.
- **DD §7C, claim 6 “integration of signals from the mobile device’s inertial measurement unit (IMU)”: NOT DIRECTLY IMPLEMENTED.** Repo `grep` for `CoreMotion|CMMotionManager|CMDDeviceMotion|SensorManager|TYPE_ACCELEROMETER|TYPE_GYROSCOPE` returns **zero direct uses for scale fusion**. The only IMU-adjacent line is `config.worldAlignment = .gravity` in `FurnitureFitARSupport.swift:90` — i.e., IMU data is consumed *transitively* by ARKit/ARCore VIO. There is no explicit Bayesian fusion step that combines IMU signals with geometric scale estimates as the patent describes.
- **DD §7B, claim 5 “door frames” as standard-dimension reference: EXISTS ONLY IN LEGACY iOS, NOWHERE ON ANDROID.**
 - `SinglePhotoRoomReconstructor.swift:265–280` uses door height **2.1 m** (not 2.0 m as the patent says): `let pixelsPerMeter = doorPixelHeight / 2.1`.
 - This is the legacy SceneKit five-plane reconstructor, NOT the production SHARP path.
 - Android has no door-as-scale-reference (verified by `grep`).
- **DD §7A, claim 3 “statistical prior over residential and commercial ceiling heights ... centered on approximately 2.4 to 3.0 meters”: PARTIAL/INACCURATE.** `RoomGeometryEngine.swift` defines `RoomGeometryConfig.standardCeilingHeightM = 2.44` — a single constant, not a probability distribution. Android `SharpRoomDimensionsV7` derives ceiling height from PLY vertex statistics with EXIF focal branching, not from a prior at all.
- **Claim 2 “the monocular image sensor is the sole depth-sensing input ... no LiDAR sensor ... is used”: FALSE FOR iOS.** `FurnitureFitARSupport.swift:91–96` actively enables LiDAR `frameSemantics.insert(.sceneDepth)` when supported, falling back to `.smoothedSceneDepth`. `FurnitureFitView.swift:1962–1966` logs `lidar=(hasLiDAR)` `depthSource=(hasLiDAR ? "sceneDepth" : "planeRaycast")`. The code does NOT refuse LiDAR;

it opportunistically uses it when present and degrades to plane raycast on non-LiDAR devices. As written, claim 2 is contradicted by the code.

- **Claim 13–14 “guiding a user ... real-time visual feedback indicating spatial coverage”:** **PARTIAL.** AR depth/coverage feedback exists for FurnitureFit overlay; a structured “scan this wall next” coverage UI as DD §2 describes is not clearly implemented in the SHARP single-photo flow (which is one capture, not a multi-frame pan).
- **Claim 16 “centimetre-level dimensional accuracy”:** **NOT VALIDATED.** No ground-truth measurement dataset exists in the repo. `scripts/sharp_percentile_sweep.py` accepts user-supplied truth via CLI flags only. iOS confidence is heuristic (0.3 / 0.5 / 0.8 in `SinglePhotoRoomReconstructor.swift`); Android `MetricScaleEstimator.kt:325–338` gates on `cv > 0.5f` — meaning it accepts scale estimates whose coefficient of variation is up to 50%. That is not centimetre-level accuracy by any honest metric.

Patent 2 — Neural Object Segmentation for Architectural Surface Identification

Verdict: NEEDS REVISION (significant)

The deployed segmentation model is **YOLO-E** (open-vocabulary instance segmentation), not the architectural-surface semantic segmentation network the patent describes. The class taxonomy is wrong, the multi-frame consistency module is missing, and the geometric regularization claims are not implemented over segmentation output.

Inaccuracies

- **DD §2 “semantic class set comprises: floor, wall (generic), wall (primary, largest visible area), wall (secondary), ceiling, door, window, furniture (generic), and background/unknown”:** **FALSE.**
 - iOS uses a large open-vocabulary `classes.json` (`Furnit/en.lproj/classes.json`, LVIS-style thousands of labels). There is no fixed `{floor, wall, ceiling, door, window, furniture}` ontology in code.
 - Android `FurnitureFitManager.kt:1815–1821` documents the production model as exporting **80 COCO classes** (`yolo-e-11l-seg-pf.onnx` exports 80 COCO classes (4 box + 80 class + 32 mask coeffs)). Code explicitly *avoids* the larger `classes.json` for IDs 0–79: “*classes.json is a larger Furnit taxonomy, so using it for class IDs 0..79 maps chairs to unrelated labels like almond*”.
 - Neither platform produces the architectural-surface class set the patent claims. Walls, floors, ceilings are NOT predicted by the segmentation network. They are derived geometrically from the SHARP point cloud (iOS `RoomGeometryEngine`, Android `SharpRoomDimensionsV7`).
- **DD §4, claims 5 “multi-frame segmentation consistency ... aggregating class probability contributions from the plurality of images for each three-dimensional surface element”:** **NOT FOUND.** This is described as “a critical innovation” of the patent; it does not exist in the code. iOS `FurnitureFitView.swift` uses an IoU-based per-frame correspondence for user-tap matching, not 3D-projected probability fusion. Android does not implement it either.
- **DD §5, claims 6–9 “geometric consistency regularization”:**
 - **Normal-based: NOT FOUND** for segmentation. iOS `RoomGeometryEngine` does use normal cues, but for room *structure* extraction from point clouds — not for biasing segmentation output.

- **MRF spatial adjacency: NOT FOUND.**
- **Room topology constraints (one floor, ≤ 1 ceiling, ≥ 1 walls): NOT FOUND** at the segmentation layer. RoomGeometryEngine does select a single floor plane via geometric heuristics, but again that's geometry from point cloud, not segmentation regularization.
- **Claim 3 “depthwise-separable convolutional operations”: NOT VERIFIABLE / THIRD-PARTY.** The deployed architecture is Ultralytics YOLOE-11L. Any depthwise-separable structure inside it is a property of the third-party model architecture; it is not a Paafekt-introduced optimization in the code. Filing this claim is risky because it implicitly claims credit for a property of a publicly available model.
- **Claim 4 “quantized to reduced-precision integer arithmetic”: FALSE.** iOS YOLO-E runs at FP32 weights with FP16 outputs. Android ONNX uses default `SessionOptions()` with no quantization, no NNAPI, no XNNPACK. INT8 in this codebase applies only to *SHARP* on Android (`ExecuTorch *_int8.pte`), not to the segmentation model.
- **Claim 10 “detecting openings within wall surfaces and classifying detected openings as door openings or window openings”: NOT FOUND.** Door/window detection only appears as a *scale reference* in the legacy iOS reconstructor, not as architectural opening output of the segmentation pipeline.
- **DD §3 “training corpus encompasses ... lighting / room types / surface materials / camera positions”:** This describes someone else’s model. The bundled YOLO-E checkpoint comes from Ultralytics; the export scripts (`scripts/yoloe_export_and_backup.py`) consume `YOLOE_PT_PATH`. There is no Paafekt-specific architectural-segmentation training corpus or trained checkpoint shipped in the repo.

Patent 3 — Centimetre-Accurate Spatial Measurement

Verdict: NEEDS REVISION (heavily — accuracy claim is the most exposed)

The “centimetre-level accuracy” claim is the most aggressive in the bundle and the least supported by the code. There is no committed ground-truth benchmark, no statistical uncertainty model with confidence intervals, and the production scale-validity gate accepts a coefficient of variation up to 50%.

Inaccuracies

- **Claim 4, abstract, DD §3 “centimetre-level accuracy”: NOT VALIDATED.**
 - No ground-truth measurement dataset exists in the repo (`/test_images/`, `/room_photos/`, `/bathroom_photos/` are JPEG photos with no associated tape-measure annotations).
 - `scripts/sharp_percentile_sweep.py` accepts truth dimensions via CLI flags (`--height`, `--depth`, `--width`) — i.e., the user supplies them ad hoc; nothing is stored.
 - iOS confidence in `SinglePhotoRoomReconstructor.swift` is a heuristic float (0.3, 0.5, 0.8), not a metric error bound.
 - Android `MetricScaleEstimator.kt`:310–338 validity gate: `if (!cv.isFinite() || cv > 0.5f) return EstimationResult(scale = 1f, isValid = false, fallbackReason = "ratio_variance_too_high")`. Accepting $cv \leq 50\%$ relative variance is the opposite of centimetre precision.
- **DD §4, claim 5 “uncertainty estimate ... confidence interval (e.g. ± 1.5 cm at 95% confidence)”: ASPIRATIONAL — NOT IMPLEMENTED.** No code computes σ or a 95% interval.

iOS has heuristic confidences. Android returns (scale, isValid, fallbackReason) — useful engineering object but not a probabilistic interval.

- **DD §3D “fitting geometric plane models to the corresponding semantic surface patches”:** **PARTIAL.** iOS RoomGeometryEngine does RANSAC plane fitting from point clouds. But measurement queries do not currently dispatch to plane-fit results; production room dimensions come from GaussianSplatView.measureRoomFromDepthBuffer (a 36×36 depth grid with quantile trimming 0.06, ≥80 points) — i.e., AABB statistics of a depth read-back, not plane fitting. Android SharpRoomDimensionsV7 similarly works from PLY vertex stats with EXIF-focal branching.
- **DD §3C “multi-frame depth fusion accuracy ... averaging out per-frame estimation errors”:** **FALSE FOR THE PRODUCTION PATH.** SHARP is single-image. There is no multi-frame averaging.
- **Claim 7 / DD §5C “room measurement summary ... dimensions and positions of all detected openings”:** **NOT FOUND.** No code path extracts door/window opening dimensions for inclusion in a room summary.
- **Claim 6 “point-and-measure user interface”:** **PARTIAL.** Two-tap calibration exists (SplatDepthQuery.TwoPointSplatCalibration — but this is *user-supplied known distance for scale*, not “tap two points and read the distance”). A point-and-measure tool does exist on Android (ArMeasureActivity.kt — taps place ARCore anchors). On iOS the equivalent is split across calibration screens.
- **Claim 8 “evaluating whether a furniture item of specified dimensions fits within an available floor space”:** **PARTIAL.** iOS has RoomFitmentMeasurement.swift and FurnitureMonocularMeasurer, but a clean “specify dimensions, check fit” workflow is not implemented end-to-end. Android has overlay-scaling for visual fit-check, not an algorithmic fit query.
- **Claim 9, DD §6 “PDF room measurement report ... structured data export (e.g., JSON or XML) ... annotated floor plan image”:** **NOT FOUND** as a MeasurementExportModule or equivalent. Sharp room metadata is persisted as PLY + JSON sidecars; that is not an exportable measurement report.

Patent 4 — AR Furniture Placement and Visualization Pipeline

Verdict: NEEDS REVISION

The placement plumbing is implemented (raycast onto floor / wall snap / drag / rotate) but the “photorealistic” rendering claims (PBR with BRDF, environment-map lighting estimation, shadow projection from estimated lights, foreground occlusion compositing) are not present in the production paths.

Inaccuracies

- **DD §2 “physically-based rendering (PBR) material parameters for each surface ... albedo, roughness, metallic, normal map, ambient occlusion map”:** **PARTIAL / OVERCLAIMED.**
 - Where furniture renders into the splat scene (iOS GaussianSplatView), the visualization is a CAShapeLayer 2D wireframe overlay over the splat, not a PBR-rendered mesh.
 - On Android, FurnitureFitOverlayView is a 2D segmentation overlay — pan/zoom/pinch — not a 3D PBR insertion.

- RealityKit does load PBR USDZ assets in some flows (Furnit/Models/USDZModelManager.swift), but the *PBR parameters are properties of the asset*, not authored by Paafekt. The patent over-claims this.
- **DD §4A “physically-based rendering pipeline ... bidirectional reflectance distribution function (BRDF) models”**: No custom BRDF implementation found. Where rendering exists at all, it uses RealityKit / SceneView / Filament defaults.
- **DD §4B, claim 7 “room lighting estimation module ... light source positions, intensities, and color temperatures ... environment map (spherical representation)”**: **NOT FOUND**. Repo `grep` for `environmentTexturing|sphericalHarmonic|reflectionProbe|ARReflection|colorTemperature|getLightEstimate|setLightEstimation` returns zero hits in .swift and .kt. The only AR environment reference is `arView.environment.sceneUnderstanding.options = [.collision, .physics]` in RealityKitView.swift:670 — i.e., collision/physics, NOT lighting estimation.
- **DD §4C, claim 8 “shadow projections from the virtual furniture items onto the room’s floor and wall surfaces ... using the estimated room lighting environment”**: **NOT FOUND**. SwiftUI `.shadow(...)` calls are 2D button decorations. `QualitySettings.shadowQuality` returns the strings “low”/“medium”/“high” (Furnit/Models/QualitySettings.swift:188–198) but the value is **never wired** to a shadow renderer in any reviewed file. There is no actual shadow generation path.
- **DD §5, claim 9 “occlusion of the furniture model by reconstructed room geometry” + “real-world foreground objects ... segmented and composited”**: **PARTIAL**. Splat geometry is the rendered scene; occlusion comes for free in that view. But the claimed foreground-occlusion-via-segmentation compositing (where YOLO-E masks the live camera and overlays the rendered furniture) is not implemented.
- **Claim 12 “accurately represents the physical scale of the furniture item within the room to within two centimetres”**: **NOT VALIDATED**. No accuracy fixtures.
- **Claim 4 “wall-snap”**: **VERIFIED** (proximity logic exists in GaussianSplatView.swift placement handlers and Android overlay positioning) — keep this claim.
- **Claim 11 “tracking ... combines IMU data with visual localization”**: **VERIFIED** transitively via ARKit/ARCore world tracking — but not by Paafekt code.

Patent 5 — On-Device Neural Architecture for Resource-Constrained Hardware

Verdict: NEEDS REVISION

The patent’s central architectural claim is a heterogeneous CPU/GPU/**NPU** scheduler that routes neural inference to the NPU. The actual code routes neural inference to the **CPU** on iOS (with explicit comments citing stability problems on GPU/ANE) and to the default-CPU ONNX path on Android. Several other supporting claims (sparse voxel grid, runtime benchmarking, interruptible refinement) are not implemented.

Inaccuracies

- **Claim 1, DD §1 “heterogeneous resource scheduler ... assign neural network inference tasks to the neural processing unit”**: **CONTRADICTED BY CODE**.

- iOS SHARPSwift:411–416: `config.computeUnits = .cpuOnly ... "FP32, computeUnits=cpuOnly".`
 - iOS YOLOModelService.swift ~205–209: `computeUnits = .cpuOnly` (per code comments, due to SIGABRT on GPU/ANE).
 - Android FurnitureFitManager.kt:126–129: `val opts = SessionOptions(); ortSession = ortEnv!!.createSession(file.absolutePath, opts)` — no NNAPI delegate, no GPU delegate, no XNNPACK delegate.
 - Both production neural workloads run on CPU. The “NPU = neural inference” claim is the OPPOSITE of what the code does.
 - **Claim 2, DD §2 “adaptive quality management module ... measure device computational capability at runtime ... benchmark”: NOT FOUND.** iOS `QualitySettings` enum (standard / high / best) appears to be set per-device-class, not via runtime throughput measurement. Android `DeviceHeuristics.isGooglePixelFamily` selects Vulkan vs CPU paths by device family — a heuristic, not a benchmark.
 - **Claim 4, DD §3 “directed acyclic graph (DAG) of processing stages” with the listed stage assignments: NOT FOUND** as a scheduler. Production code uses serial throttling (`arSessionDelegateHeavyMinInterval = 0.28` in iOS; `minFrameIntervalMs = 55L` in Android) and a “drop-frames coalesce-to-latest” pattern (`pendingLatestSegmentationFrame`).
 - **Claim 5, DD §4A “sparse voxel representation ... hash map indexed by voxel coordinates”: NOT FOUND.** There is no voxel grid in the production reconstruction (which is 3D Gaussian splat → PLY).
 - **DD §4B “tiled processing ... store back to device storage before the next tile is loaded”: MISLEADING.** ExecuTorch SHARP has `part4b_tile_*.pte` files (`models_cpu/sharp_split_part4b_tile_*.pte`) — these are *neural-inference tiles* (model split into chunks for memory reasons), not the reconstruction-output tiling described in the patent.
 - **DD §4C “progressive level of detail ... loaded progressively for the spatial region currently in view”: NOT FOUND.**
 - **DD §4D, claim 6 “memory pressure monitoring ... selectively release cached data”: PARTIAL.** iOS has `releaseInferenceMemoryAfterGeneration`, YOLO `releaseResources`, and a < 2 GB physical memory gate in `SHARPSwift.loadModel`. This is closer to a one-shot RAM check + post-inference release than the real-time pressure-driven cache eviction the patent describes. Honest characterization needed.
 - **Claim 9, DD §6 “interruptible refinement ... resume from current state”: NOT FOUND.** SHARP runs as a single asynchronous batch; there is no incremental refinement loop to interrupt.
 - **DD §5C “knowledge distillation”: NOT FOUND** in any deployed runtime artefact. (May be a training-time concept used during model preparation; not an attribute of the deployed system.)
 - **DD §5D “operator fusion combining adjacent neural network operations into single hardware-efficient kernels”: PARTIAL.** Conv+BN fuse is documented in `android/pyfiles/export_sharp_executorch_fp16.py` via `torch.ao.quantization.fuse_modules`. Honest.
 - **Claim 8 “8-bit or 4-bit integer representation”: PARTIAL.** Android SHARP genuinely has INT8 (`ExecutorchInt8Sharp`, files like `models_cpu/sharp_split_part1_int8.pte` ~151 MB). iOS has none. The patent claim is true *for one platform, for one model*. Should be scoped down or honestly disclosed as such.
-

Patent 6 — Integrated End-to-End Furniture Visualization System

Verdict: PASS WITH MINOR ISSUES

This patent describes the product system at a higher level of abstraction. The integration of scanning → reconstruction → measurement → catalog → placement → visualization → persistence into one app is genuinely what's built. A few specific sub-claims do not have code support and should be removed or softened.

Inaccuracies

- **Claim 5, DD §8 “commerce integration subsystem ... in-application purchase flow”: NOT FOUND.** No purchase intent, retailer hand-off, or in-app payment surface in the reviewed Swift/Kotlin. Should be removed as a claimed component for now (or carved into a future continuation).
- **DD §7, claim 4 “locally-stored furniture catalog subset enabling offline browsing ... network-accessed extension providing the full catalog”: PARTIAL.** Catalog is hardcoded in Views/Components/SharpRoomFurnitureSupport.swift. No “network-accessed catalog extension” code path was found.
- **Claim 12, DD §9D “undo/redo history of furniture placement actions”: NOT FOUND** as a structured undo stack.
- **Claim 11, DD §10 “shared platform-independent spatial computing layer ... single implementation of the core algorithms to execute on both platforms”: MISLEADING.** What is shared is the **SHARP model** (Apple’s SHARP architecture, exported separately as Core ML for iOS and ExecuTorch for Android). The **code that calls the model and processes its output is duplicated** in Swift and Kotlin (e.g., Furnit/Services/OnDevice/SHARPService.swift vs android/.../services/SharpService.kt; SinglePhotoRoomReconstructor.swift exists in both languages with parity comments). The “single implementation of the core algorithms” claim is false; “shared model artefact with platform-specific orchestration code” is the truthful version.
- **DD §2 timing claims “scanning session typically lasting 30 to 90 seconds” and “reconstruction completes within approximately 30 to 120 seconds”: NOT VALIDATED** (no benchmark in repo). SHARP is a *single-image* capture — not a 30–90 s scanning session. Reconstruction is one async pass; 30–120 s might be plausible on mid-range hardware but it’s not measured here.
- **Claim 6 “unified internal data model ... data written by one module is accessible to all other modules without manual transfer”: TRUE** — RoomModel, SharpRoomFurnitureItem, RoomFitmentMeasurement etc. are shared types within each platform. Keep.
- **Claim 7 “do not transmit room photographic data or spatial model data to external servers”: VERIFIED.** Repo grep returns zero URLSession / OkHttp / Retrofit in the spatial pipeline. Firebase is used for *authentication only* (AuthenticationManager, FurnitApp.swift) — not for spatial data. This claim is well-supported. Keep.
- **Claim 8 “all core functionality ... operates without network connectivity”: VERIFIED** for the spatial pipeline. Keep.
- **Claim 10 “multi-session capability”: VERIFIED.** Sharp room PLY persistence exists in sharp_rooms/room_<timestamp>/ (Android) and equivalent iOS SHARPModels storage. Keep.

3. Gap Report — Methods in Code Not Described in Any Patent

Each entry is written as ready-to-paste prose. Insert it under the relevant patent's **Detailed Description** before filing. Filing today without these means they cannot appear in the corresponding non-provisional twelve months from now.

GAP-1 — Single-image neural Gaussian-splat reconstruction as a primary embodiment

Where: `Furnit/Services/OnDevice/SHARPService.swift`, `android/app/src/main/java/com/furnit/android/services/ExecutorchInt8Sharp.kt`, `services/SharpService.kt`. **Belongs in:** Patent 1 (alternative embodiment) and Patent 6 (architecture description).

Why novel: The patent's headline embodiment is multi-view photogrammetry. The actual production embodiment is a single-image neural model that emits 3D Gaussian splat primitives from one captured photograph and one set of camera intrinsics, executed entirely on-device. This is a different invention from the one currently described, and right now it is described nowhere in the bundle.

Insertable description (ready to paste):

In an alternative embodiment, the on-device three-dimensional room reconstruction is performed not from a multi-view image sequence but from a single captured photograph of the interior room, processed through a neural inference model that directly emits a parameterised three-dimensional point primitive representation of the scene without an intermediate per-frame depth map or multi-view triangulation step. The neural inference model accepts as input a single rectangular colour image at a fixed input resolution (in one embodiment, 1536×1536 pixels) together with the focal length and principal point of the capturing camera (obtained from EXIF metadata or platform camera intrinsics), and emits as output a plurality of parameterised three-dimensional primitives, each primitive having an associated three-dimensional position, an anisotropic scale tensor, an orientation, a per-primitive colour, and an opacity. Said primitives are persisted to a structured binary file (in one embodiment, a Polygon File Format file with a fixed number of attributes per vertex) that may subsequently be rendered in real time on the same mobile device using a graphics-processing-unit splat renderer.

The single-image embodiment exchanges the multi-view metric-consistency property of the multi-view embodiment for substantially reduced capture friction (a single shutter press in place of a thirty-second guided pan), and is preferred in user contexts where capture time is the primary constraint. Metric scale for the single-image embodiment is recovered through any one or more of: integration of platform-supplied augmented-reality camera tracking distance estimates obtained at the moment of capture; a user-supplied calibration step in which the user marks two points on a known-distance feature in the captured image; and statistical extraction of architectural plane separations from the resulting three-dimensional primitive cloud combined with a prior over residential ceiling heights.

GAP-2 — Hybrid metric-scale estimation: AR-tracking-derived anchors fused with neural monocular depth via coefficient-of-variation gating

Where: android/app/src/main/java/com/furnit/android/services/MetricScaleEstimator.kt:280–340, ar/FurnitureFitArMetrics.kt. **Belongs in:** Patent 1 (scale-disambiguation alternative) and Patent 3 (accuracy source).

Why novel: The patent’s scale-disambiguation section enumerates floor-ceiling parallelism, door-frame priors, and IMU integration. The actually shipped scale estimator does something different: it samples *metric anchors* from the platform AR depth API (which is itself a tracker-internal estimator), pairs them with the per-pixel monocular depth values produced by the neural model at the same image coordinates, computes a per-pair scale ratio, takes a robust statistic of the ratios, and only emits a final scale estimate when the coefficient of variation across the ratio set is below a configurable threshold. This is a two-source consensus mechanism, not a single-source estimate, and the cv-gate gives it a principled way to refuse rather than emit a wrong number.

Insertable description:

In one embodiment, the metric scale of the on-device three-dimensional reconstruction is recovered by a two-source consensus procedure. A first source produces a sparse set of metric distance anchors derived from the platform’s augmented-reality camera tracking subsystem, each anchor comprising a two-dimensional image coordinate and a metric distance in metres at said coordinate; said anchors may be sampled from any of: a depth image produced by the platform AR depth subsystem, a feature-tracked anchor created by the platform world-tracking subsystem, or a plane intersection raycast onto a tracked horizontal or vertical plane. A second source produces, for the same image, a dense per-pixel monocular depth estimate from the neural inference model that also produces the three-dimensional primitive reconstruction, said per-pixel estimate being affine-invariant (i.e., correct up to an unknown multiplicative scale factor and additive offset).

For each metric anchor, the system samples the corresponding monocular depth value at the same image coordinate and computes a ratio between the metric anchor distance and the monocular depth value. The set of per-anchor ratios is reduced to a single robust scale estimate (in one embodiment, the median or a trimmed mean) and a coefficient of variation across the ratio set. If said coefficient of variation exceeds a configurable threshold (in one embodiment, 0.5), the procedure reports a non-validity signal and a fallback reason rather than emitting a scale value, on the basis that the two sources are insufficiently consistent to support a metric measurement claim. If said coefficient of variation is below the threshold, the robust scale estimate is applied globally to the reconstruction.

This two-source consensus mechanism, with its explicit refusal-to-estimate gate, is structurally different from a single-source scale resolver in that it provides a defensible engineering criterion for when the system declines to produce a measurement at all, rather than emitting an unreliable measurement that the user cannot distinguish from a reliable one.

GAP-3 — User-supplied two-point calibration for monocular metric scale recovery

Where: `Furnit/Services/SplatDepthQuery.swift` (`TwoPointSplatCalibration`). **Belongs in:** Patent 3 (alternative scale source) and Patent 1 (scale-disambiguation §7 alternative).

Why novel: The patent positions itself against “manual placement of scale references” by the user. The actual code includes such a fallback as one of several available scale sources. Filing without describing it leaves the option closed off in the non-provisional.

Insertable description:

In one embodiment, when the automatic metric-scale recovery procedures fail to converge or are unavailable on the executing hardware, the system provides an interactive two-point calibration interface in which the user is presented with a rendered view of the reconstructed three-dimensional scene and selects two displayed points that the user knows to correspond to a known physical distance (for example, the width of a door, the height of a kitchen worktop, or the length of a tile). The user enters the known physical distance in their preferred unit system. The system computes a metric scale factor as the ratio of the known physical distance to the measured distance between the two selected points in the reconstruction’s internal coordinate units, and applies said scale factor globally to the reconstruction. The two-point calibration is preserved with the persisted reconstruction so that subsequent measurement queries against the same reconstruction use the calibrated scale without further user input.

GAP-4 — EXIF-focal-length-conditioned dimension extraction with shot-type branching

Where: `android/app/src/main/java/com/furnit/android/Utils/SharpRoomDimensionsV7.kt` (`measureBest`, `STRAIGHT / CORNER / FALLBACK_NO_FOCAL` branches). **Belongs in:** Patent 3 (measurement extraction).

Why novel: The system reads camera focal length from the EXIF metadata of the captured image, then branches its dimension-extraction algorithm depending on (a) whether focal length is available and (b) whether the captured photograph is a “straight-on” view of one wall versus a “corner” view of two adjacent walls. This is a non-obvious robustness mechanism that is not described anywhere in the patents.

Insertable description:

In one embodiment, the dimension-extraction subsystem reads camera intrinsic parameters from the metadata of the captured photograph, said metadata comprising at least a recorded focal length value and an image sensor pixel pitch. The dimension-extraction algorithm operates in one of a plurality of disjoint branches selected at runtime based on whether the recorded focal length value is present and within a plausible range, and based on a classification of the photograph’s view geometry as either a substantially perpendicular view of a single wall plane (a ‘straight-on’ shot) or a view containing two adjacent wall planes meeting at an interior corner (a ‘corner’ shot). The straight-on branch derives wall width from the lateral extent of points

classified as belonging to the dominant wall plane, multiplied by a depth value at the geometric centre of said wall plane and divided by the recorded focal length. The corner branch derives the width of each visible wall plane separately by projecting onto the plane's surface normal. Where focal length is absent or implausible, a fallback branch derives dimensions from the relative extents of the three-dimensional primitive cloud combined with a ceiling-height prior, and reports a reduced confidence value on the resulting dimensions.

GAP-5 — Memory-budget-conditioned model loading with explicit refusal

Where: `Furnit/Services/OnDevice/SHARPService.swift` (~382–386, < 2 GB physical memory gate); release/reload pattern with `releaseInferenceMemoryAfterGeneration`. **Belongs in:** Patent 5 (memory management).

Why novel: The patent describes “memory pressure monitoring” as a passive eviction mechanism. The actual code goes further: it refuses to load the heavy reconstruction model at all when total physical memory falls below a hard threshold, returning a structured error to the application layer that allows the application to surface a graceful “this device is below the minimum specification for this operation” message instead of attempting and crashing. Combined with the post-inference deterministic release of the model from RAM, this is a different design pattern from the patent’s pressure-monitor-and-evict description.

Insertable description:

In one embodiment, the on-device neural inference subsystem implements a hard pre-load memory gate that queries the total physical memory of the executing device at model-load time and refuses to load a high-memory neural inference model when the total physical memory falls below a configurable threshold (in one embodiment, two gigabytes). On refusal, the subsystem returns a structured error code to the calling application layer indicating insufficient memory rather than attempting the load and risking an out-of-memory termination. The application layer surfaces an informative message to the user and, where applicable, offers a degraded alternative procedure that does not require the high-memory model. Following successful inference, the subsystem deterministically releases the loaded model and its working memory back to the operating system rather than retaining it for potential reuse, on the basis that retaining a multi-gigabyte model in resident memory between user actions is more harmful to overall device responsiveness than incurring the load latency on the next inference request.

GAP-6 — Dual neural inference backends behind a single capability-detected facade

Where: `Android services/FurnitureFitManager.kt` (ONNX Runtime path) + `services/NcnnYoloE.kt, services/YoloEImageInference.kt` (NCNN path); flavour selection based on `DeviceHeuristics.isGooglePixelFamily` and `BackendConfig`. **Belongs in:** Patent 5 (heterogeneous execution) and Patent 6 (architecture).

Why novel: The Android implementation maintains two complete inference backends for the same model family — ONNX Runtime for one set of consumer paths and NCNN for another — and

selects between them based on device-family heuristics and backend-configuration flags. This is a redundancy-for-portability design specific to Android's hardware fragmentation.

Insertable description:

In one embodiment of the on-device neural inference subsystem implemented for an operating system that targets a hardware ecosystem with substantial inter-vendor fragmentation in neural-acceleration capabilities, the same trained neural model is exported to a plurality of distinct on-device inference runtime formats and bundled with the application. At application startup or on first use of a given inference path, a backend-selection module evaluates one or more of: device manufacturer and model family identifiers, presence of specific co-processors and their drivers, and historical inference latency observed on the device, and selects one of the bundled runtime formats for use on the executing device. This redundancy permits the application to operate at acceptable performance across a wider range of consumer devices than any single runtime would accommodate, at the cost of increased application binary size.

GAP-7 — Manual room boundary capture as a non-ML fallback to neural reconstruction

Where: android/app/src/main/java/com/furnit/android/RoomBoundaryActivity.kt, services/SinglePhotoRoomReconstructor.kt, services/GlbGenerator.kt; iOS counterpart in Services/RoomReconstruction/SinglePhotoRoomReconstructor.swift. **Belongs in:** Patent 1 (alternative embodiment).

Why novel: When the neural reconstruction is unavailable or undesired, the application provides a manual boundary-capture interface in which the user drags two-dimensional polylines over a photographed room to denote the visible floor / ceiling / wall edges, and the system extrudes those boundaries into a textured five-plane three-dimensional room model. This is an “ML-free” reconstruction path that produces a downstream-compatible room model, and it is not described in the patents.

Insertable description:

In one embodiment, the system provides an alternative manual reconstruction procedure that does not rely on neural inference. The user is presented with the captured photograph of the interior room and a set of draggable two-dimensional polyline overlays representing the floor edge, the ceiling edge, and any number of wall-edge boundaries visible in the photograph. The user adjusts the polyline endpoints to align with the visible structural edges. From the user-adjusted polylines, the system constructs a three-dimensional room model by treating each visible wall-floor and wall-ceiling polyline as the projection of a horizontal architectural edge of standard height (using a configurable ceiling-height prior or a user-supplied ceiling height), extruding the wall surfaces vertically between said horizontal edges, and texturing each generated wall surface with the corresponding pixels of the captured photograph. The resulting three-dimensional room model is structurally compatible with the model produced by the neural reconstruction procedure and may be consumed interchangeably by the downstream measurement, furniture-placement, and visualisation subsystems.

GAP-8 — Camera contention detection and dual-camera arbitration

Where: iOS `Furnit/Views/FurnitureFit/FurnitureFitView.swift` and supporting AR/AVCapture handling — `figCameraContentionErrorCodes` checks. **Belongs in:** Patent 4 (placement / live overlay) or Patent 6 (architecture).

Why novel: The application maintains both an ARKit session (for tracked-world placement) and an AVCapture session (for the live segmentation overlay) and detects when iOS reports camera-resource contention between the two, then arbitrates between them at runtime — pausing one session, reconfiguring the other, and resuming. This is a non-obvious robustness mechanism for the dual-pipeline architecture, and it is not described.

Insertable description:

In one embodiment, the application maintains two concurrent camera-consuming subsystems on the same device: an augmented-reality session that consumes the camera as a tracked-world input and a still-or-streaming capture session that consumes the camera as a colour-image source for neural segmentation. Because the operating system enforces at-most-one exclusive consumer of the camera hardware at certain configurations, the application includes a contention-detection module that monitors operating-system error codes indicative of camera-resource contention and, on detecting contention, arbitrates between the two subsystems by suspending the lower-priority subsystem, reconfiguring the higher-priority subsystem with a compatible camera configuration, resuming the lower-priority subsystem when possible, and surfacing an informative message to the user if both subsystems are required and cannot coexist on the executing device.

GAP-9 — Quantile-trimmed depth-grid AABB room measurement

Where: `Furnit/Views/Components/GaussianSplatView.swift` — `measureRoomFromDepthBuffer` (36×36 grid, trim 0.06 quantiles, ≥ 80 inlier points). **Belongs in:** Patent 3 (measurement extraction).

Why novel: Production room dimension extraction from the rendered splat scene operates by reading back the depth buffer, sampling it on a 36×36 grid, trimming the 6th and 94th percentiles to reject outliers from sky, holes, and over-near surfaces, requiring at least 80 inlier samples to emit a measurement, and computing an axis-aligned bounding box over the surviving samples. This is a measurement extraction *from the renderer*, not from the geometric model directly, and it is not described.

Insertable description:

In one embodiment, room dimensions are extracted from the rendered three-dimensional scene rather than from the raw geometric model. The graphics-processing-unit depth buffer associated with a current view of the rendered scene is sampled at a regular grid of view-coordinate positions (in one embodiment, a 36-by-36 grid). Each grid sample is unprojected from view coordinates to scene coordinates using the inverse of the current view-and-projection transform. The unprojected sample set is filtered by trimming a configurable lower and

upper quantile of depth values (in one embodiment, 6 percent at each tail) to reject outlier samples corresponding to background pixels, scene-geometry holes, and surfaces nearer than the camera near-plane. A minimum-sample-count gate (in one embodiment, 80 surviving samples) is enforced before emitting a measurement; below the gate, the system reports an insufficient-coverage signal and prompts the user to adjust the viewpoint. From the surviving samples, an axis-aligned bounding box is computed in scene coordinates and reported as the measured extents of the visible portion of the reconstructed room.

GAP-10 — Bundle-id-and-on-demand-resource hybrid model delivery

Where: Furnit/Services/OnDevice/SHARPService.swift (NSBundleResourceRequest for SHARP), YOLOModelService.swift (model search path traverses bundled .mlmodelc then .mlpackage then on-demand-resource location). **Belongs in:** Patent 5 or Patent 6 (model packaging / installation).

Why novel: The 1.2 GB SHARP model is too large to ship in the base application binary on the iOS App Store (which has size limits and incurs cellular-download penalties at certain thresholds). The application uses Apple's On-Demand Resources mechanism to deliver the heavy model post-install, with a fallback that traverses multiple bundled formats (.mlmodelc compiled, .mlpackage source) before resolving to the on-demand path. This packaging strategy is not described.

Insertable description:

In one embodiment, the heaviest neural inference model used by the on-device reconstruction subsystem is delivered to the executing device as a post-installation resource fetch rather than as a component of the base application binary, on the basis that said model's size (in one embodiment, in excess of one gigabyte) exceeds the size budgets imposed by the application distribution store on the application binary itself. The application includes a model-resolution module that, on first use of the reconstruction subsystem, traverses a configurable search path of model locations, including a compiled-model location within the application bundle, a source-model-package location within the application bundle, and an on-demand-resource location managed by the operating system's resource-fetch service, and uses the first matching location found. If the on-demand-resource location is selected and the resource is not yet locally cached, the model-resolution module triggers a resource fetch and surfaces a download-progress indicator to the user.

4. Surgical Claim Fixes (Quote Old → Suggest New)

The principle: the safest fix for a claim that is broader than what the code does is to *narrow* the claim to what the code actually does, then add the broader version as a dependent claim if you want it on the record. Removing false claims is far less risky than leaving them in.

Patent 1

P1-Claim 2 (current): > “The method of claim 1, wherein the monocular image sensor is the sole depth-sensing input to the method, and wherein no LiDAR sensor, structured-light sensor, time-of-flight sensor, or stereoscopic camera pair is used.”

Issue: Directly contradicted by FurnitureFitARSupport.swift:91–96 which inserts .sceneDepth (LiDAR) when supported.

Suggested replacement: > *“The method of claim 1, wherein the method does not require a LiDAR sensor, structured-light sensor, time-of-flight sensor, or stereoscopic camera pair to produce the three-dimensional surface representation, and wherein, on mobile computing devices that include such an additional depth-sensing component, the method optionally consumes signals from said additional component as a non-essential refinement input to improve depth-estimate density or scale-estimate confidence without changing the method’s reliance on the monocular image sensor as the primary depth-sensing input.”*

P1-Claim 1 (current independent claim — captures only the multi-view embodiment): Add a SECOND independent claim that captures the single-image SHARP embodiment so that the priority date covers both:

Suggested addition (P1-Claim 1A, new independent): > *“A computer-implemented method for three-dimensional room reconstruction, comprising: capturing, using a monocular image sensor of a mobile computing device, a single image of an interior room; reading camera intrinsic parameters associated with said single image; executing, on the mobile computing device, an on-device neural inference model that accepts as input said single image and said camera intrinsic parameters and emits as output a plurality of parameterised three-dimensional point primitives, each primitive having an associated three-dimensional position, a scale tensor, an orientation, a colour, and an opacity; persisting said primitives to a structured file in non-transitory storage of the mobile computing device; resolving a metric scale of the resulting three-dimensional representation using at least one of: an augmented-reality-tracking-derived metric distance estimate sampled at the time of capture, an architectural-plane separation extracted from the resulting three-dimensional representation combined with a statistical prior, and a user-supplied two-point calibration on the persisted three-dimensional representation; and outputting a metric-scaled three-dimensional model of the interior room; wherein all steps execute entirely on the mobile computing device without transmission of image data to an external server.”*

P1-Claim 3 (current floor-ceiling prior): > *“The method of claim 1, wherein resolving the metric scale comprises detecting a floor surface and a ceiling surface in the three-dimensional surface representation, measuring a separation distance between the floor surface and the ceiling surface in internal reconstruction units, and applying a statistical prior distribution over room height dimensions to determine a metric scale factor.”*

Issue: Code uses a single ceiling-height constant (2.44 m), not a probability distribution.

Suggested replacement (broader, covers both): > *“The method of claim 1, wherein resolving the metric scale comprises detecting a horizontal floor plane and a horizontal ceiling plane in the three-dimensional surface representation, measuring a vertical separation between said planes in internal reconstruction units, and applying a configured ceiling-height value or a probability distribution over residential and commercial ceiling heights to determine a metric scale factor.”*

P1-Claim 5: > *“The method of claim 4, wherein the architectural features of standard known dimensions comprise door frames.”*

Issue: Door reference is 2.1 m (not 2.0 m as the patent body says) and only exists in the legacy iOS reconstructor — Android has no door reference at all. Either soften or remove. Recommended: keep the dependent claim but soften the description to read “approximately 2.0 to 2.1 metres” so the body matches one or the other code path.

P1-Claim 6 (IMU integration): The claim is technically defensible because ARKit/ARCore consume IMU under the hood for the world-tracking that produces the AR depth and anchor signals consumed by the scale estimator. But the *direct* IMU-fusion-with-geometric-scale step described in DD §7C is not implemented. Soften the description so that “IMU integration” reads as “either directly via system inertial-sensor APIs or transitively via consumption of platform augmented-reality tracking outputs that themselves consume inertial-sensor signals.”

Patent 2

P2-Claim 4: > *“The method of claim 1, wherein the neural segmentation model is quantized to reduced-precision integer arithmetic for execution on mobile neural processing hardware.”*

Issue: False for the deployed segmentation model on both platforms.

Suggested replacement: > *“The method of claim 1, wherein the neural segmentation model is selected from a plurality of model variants of differing numerical precision and computational footprint, and wherein the model variant deployed on a given mobile computing device is selected based on at least one of: the device’s available compute units, the device’s available system memory, and an observed runtime stability of higher-precision execution paths on said device.”*

P2-Claim 1 + DD §2 ontology: The fixed {floor, wall, ceiling, door, window, furniture} ontology should be removed from the *primary* embodiment description and replaced with an open-vocabulary characterization:

Suggested replacement of class-set claim language: > *“... wherein the neural segmentation model emits per-pixel class labels drawn from a class vocabulary, said class vocabulary comprising at least a furniture class group and at least one ground-truth class for each visible architectural surface category that the model has been trained to discriminate; and wherein in one embodiment the class vocabulary is a closed set comprising at least floor, wall, ceiling, door opening, window opening, and one or more furniture categories, and in another embodiment the class vocabulary is an open vocabulary supporting hundreds or thousands of distinct labels including non-architectural object categories.”*

P2-Claim 5 (multi-frame consistency fusion): This is described as the “critical innovation” of Patent 2. It is not implemented. Two options:

- **Option A (defensible):** Remove the claim from this provisional. File it later as a continuation-in-part if and when implemented.

- **Option B (aggressive):** Keep the claim, on the basis that a provisional describes the invention and a non-provisional has 12 months to implement before priority claim. *Risk:* if it never gets implemented, the non-provisional claim will lack written-description support and is vulnerable.

Recommendation: Option A. Do not file claims to unimplemented behaviour as the primary novel contribution.

Patent 3

P3-Claim 4: > *“The method of claim 1, wherein the measurement value achieves centimetre-level accuracy without use of a LiDAR sensor, structured-light sensor, time-of-flight sensor, or stereoscopic camera pair.”*

Issue: Unvalidated, and the LiDAR exclusion is contradicted by the iOS code as for Patent 1.

Suggested replacement (defensible): > *“The method of claim 1, wherein the measurement value is reported together with a per-measurement validity indicator computed from a coefficient-of-variation metric over the metric-scale estimation procedure, and wherein the system declines to emit a measurement value when said coefficient of variation exceeds a configurable threshold; and wherein the method does not require a LiDAR, structured-light, or time-of-flight sensor to compute said measurement, but, on mobile computing devices that include such a sensor, the method optionally consumes signals from said sensor as a non-essential input to said metric-scale estimation procedure.”*

Drop the standalone “centimetre-level accuracy” assertion as a *claim*. Move the centimetre-level statement into the SUMMARY/DETAILED-DESCRIPTION as an aspirational target supported by certain embodiments under specified conditions, not as a hard claim.

P3-Claim 5: Replace heuristic-confidence wording with the actual cv-gating mechanism (which is concrete, novel, and defensible): > *“... wherein the uncertainty estimate comprises a coefficient-of-variation value computed across a plurality of independent metric-scale-ratio observations, said coefficient-of-variation value being compared against a configurable validity threshold, and wherein the measurement is reported as not-valid when said value exceeds said threshold.”*

P3-Claim 9 (PDF/JSON/XML export): Either remove or downgrade to “in one embodiment, the room model and computed measurement values are persisted to a structured file in non-transitory storage of the mobile computing device for subsequent retrieval and display.” The PDF report module does not exist in code.

Patent 4

P4-Claim 6 (PBR pipeline): > *“... wherein rendering comprises implementing a physically-based rendering pipeline using estimated room lighting conditions.”*

Issue: No estimated lighting conditions; no custom PBR; the splat path renders Gaussian primitives with no PBR at all.

Suggested replacement: > *“... wherein the composite visualisation is rendered through a graphics-processing-unit rendering pipeline that consumes pre-authored material parameters associated*

with the selected furniture item, said material parameters being read from the furniture catalog and applied at render time without modification by the rendering pipeline.”

Then add a SEPARATE dependent claim that covers the lighting-estimation embodiment as something the system *may* do in one embodiment, without claiming it is implemented in the base embodiment.

P4-Claims 7 + 8 (lighting estimation + shadow projection): Both unimplemented. Either remove or move to a separate continuation. Same recommendation as P2-Claim 5.

P4-Claim 12 (“within two centimetres”): Drop the specific “two centimetres” number from the claim. Replace with: *“... wherein the composite visualisation places the furniture item at the same metric scale as the surrounding reconstructed room model, such that a furniture item of stored width X centimetres occupies an extent in the visualisation that subtends the same angular field as a real object of width X centimetres at the same position.”*

This is a *relational* accuracy claim (the furniture is at the same scale as the room) which is what the code actually does, and avoids the unvalidated “two centimetres” number.

Patent 5

P5-Claim 1 (heterogeneous scheduler with NPU = neural inference): > *“... a heterogeneous resource scheduler ... configured to: assign neural network inference tasks to the neural processing unit ...”*

Issue: The opposite of what the code does (production iOS runs neural inference on CPU).

Suggested replacement (much narrower, defensible): > *“... a workload-routing module configured to: select for each neural inference task a target compute resource from a set comprising at least the central processing unit and, where available, the neural processing unit and the graphics processing unit; wherein said selection is conditioned on at least one of: an observed runtime stability of executing said inference task on each candidate compute resource on the executing device, a configured per-resource permission, and an observed end-to-end inference latency measured during prior executions of said inference task on the executing device; and wherein said selection may, on a given device, route a given neural inference task to the central processing unit even where the neural processing unit is available, on the basis that prior execution on the neural processing unit has been observed to produce errors or unacceptable variance in inference latency.”*

That language is narrower than the original, but it accurately describes what the code does (cpuOnly with explicit-stability-driven choice, not naive NPU routing) and gives you a defensible, novel mechanism.

P5-Claim 5 (sparse voxel hash map): Not implemented. Remove or move to a continuation.

P5-Claim 9 (interruptible refinement): Not implemented. Remove or move to a continuation.

P5-Claim 8 (4-bit / 8-bit quantization): Restrict to “at least one of FP16 and INT8 representations” and qualify “for at least one of the neural models in the spatial computing pipeline” — so that the truthful Android-SHARP-INT8 fact supports the claim without overstating the iOS path.

Patent 6

P6-Claim 5 (commerce integration): Not implemented. Recommended: remove from this provisional (file the commerce embodiment in a follow-up provisional once an actual commerce surface exists).

P6-Claim 11 (shared platform-independent spatial computing layer): Replace with: > “... implemented for both an iOS operating system and an Android operating system, with at least one neural inference model artefact derived from a common upstream model architecture and exported separately into the on-device inference runtime format native to each operating system, and with platform-specific orchestration code that calls said model artefacts and processes their outputs.”

That is the truthful “shared model artefact, duplicated orchestration code” framing.

P6-Claim 12 (undo/redo): Not implemented. Remove.

5. Loophole Check — Competitor Workarounds and How to Close Them

For each patent, the most obvious competitor evasion strategies and the surgical claim language that closes them.

Patent 1

- **Loophole 1 — Skip the multi-view step entirely:** A competitor implements the single-image neural reconstruction (which is what *this codebase* does) and argues your independent claim does not read on it because you require “a plurality of images.” **Fix:** Add the proposed second independent claim P1-Claim 1A above (the single-image embodiment).
- **Loophole 2 — Replace the on-device neural model with a “client-side feature extraction + server-side reconstruction” hybrid:** A competitor extracts visual features on-device, sends only the features to a server, and receives the reconstruction back; argues that no *image data* was transmitted (only features). **Fix:** Replace “without transmission of image data to an external server” with “without transmission to an external server of any data from which the captured imagery or the reconstructed three-dimensional representation can be reconstructed.”
- **Loophole 3 — Use a learned scale estimator rather than the architectural-feature scale resolver:** A competitor trains a neural network that directly emits an absolute-scale 3D model end-to-end. **Fix:** Add a dependent claim covering learned-scale-estimation: “... wherein resolving the metric scale comprises executing an on-device neural inference model that accepts as input the affine-invariant three-dimensional representation and emits as output a single scalar metric scale factor.”

Patent 2

- **Loophole 1 — Use 2D segmentation only (no 3D label propagation):** A competitor runs per-frame segmentation, ignores the 3D label step (which is your “critical innovation”), and argues their system does not infringe. **Fix:** Either drop the 3D label propagation as a *required* step in

claim 1 (so a 2D-only system still infringes) — or, conversely, keep it and accept that 2D-only systems do not infringe. You cannot have both. Decide consciously.

- **Loophole 2 — Open-vocabulary segmentation instead of a fixed architectural ontology:** A competitor uses an open-vocabulary segmenter (this is what your own iOS code does!) and argues their system does not predict the patent's {floor, wall, ceiling, door, window, furniture} ontology. **Fix:** Adopt the open-vocabulary class-set language proposed under P2 above so that *both* fixed and open ontologies fall within the claim.
- **Loophole 3 — Move geometric regularization to a separate module:** A competitor runs YOLO-style segmentation and a separate geometric room-fitter, and argues that their per-pixel segmentation output is not “regularized” because the regularization happens in a separate module. **Fix:** Make claim 6 (geometric consistency regularization) read on the *combination* of segmentation output with separate geometric processing, not on regularization *internal* to the segmentation model.

Patent 3

- **Loophole 1 — Compute measurements without a confidence interval:** A competitor measures rooms, omits the uncertainty output entirely, and argues claim 5 is therefore not infringed. **Fix:** Make the uncertainty step optional in the independent claim and add it back as a dependent claim. This widens the scope of the independent claim.
- **Loophole 2 — Use a known reference object held by the user:** Patent 3 explicitly disclaims user-held reference objects. A competitor that reverts to that approach is outside your claim. **Fix:** This is intentional disclamation and probably should stay; if you want to cover it, file a separate continuation that covers user-held references as one optional source.
- **Loophole 3 — Cloud-assisted measurement that runs on-device when offline:** A competitor's system runs in the cloud by default and falls back to on-device only when offline; claims to be “primarily cloud” so escapes “executes entirely on-device.” **Fix:** Add language to the independent claim that the on-device path is the *default* execution path, not a fallback path: “... wherein the metric-scale estimation procedure executes on the mobile computing device as its default and primary execution mode and does not require external connectivity to produce a measurement.”

Patent 4

- **Loophole 1 — Re-render furniture catalog with stored materials but no estimated lighting:** A competitor uses pre-authored materials and ignores room lighting estimation entirely (this is what your own code does!). They escape claims 6–8. **Fix:** Move lighting estimation into a dependent claim. Make the independent claim cover scale-accurate placement with stored material parameters as the base case.
- **Loophole 2 — 2D overlay rather than 3D placement:** A competitor renders furniture as a 2D image overlay scaled by depth at the touch point (this is what your Android FurnitureFit does!). They argue they do not “place the three-dimensional model” because no 3D mesh insertion occurs. **Fix:** Add a dependent claim that explicitly covers a 2D overlay computed by projecting the furniture item's silhouette at the placement-position depth.
- **Loophole 3 — Place furniture without snap-to-wall:** A competitor omits wall snapping. **Fix:** Snap-to-wall should be a dependent claim, not a required step.

Patent 5

- **Loophole 1 — Run neural inference on CPU only:** A competitor uses CPU-only inference (this is what your own iOS code does for stability!). They escape the “assign neural network inference to NPU” claim. **Fix:** As proposed, replace the NPU-routing claim with the workload-routing-with-stability-conditioning claim.
- **Loophole 2 — Use a single uniform model for all hardware:** A competitor ships one model for all devices and argues no “adaptive quality management” is performed. **Fix:** Make adaptive quality a dependent claim, not a feature of the independent claim.
- **Loophole 3 — Use cloud bursting for heavy frames:** A competitor processes 90% of frames on-device and offloads complex frames to the cloud. **Fix:** Add explicit “without offloading any portion of the spatial computing pipeline to an external server during a measurement or reconstruction session.”

Patent 6

- **Loophole 1 — Charge for some features, free for others:** A competitor unbundles the workflow into multiple apps that the same publisher offers (one for scanning, one for placement). They argue no “integrated” system exists. **Fix:** Define “integrated” in the claim as “executable as a single application binary” or “presented to the user as a single workflow” so that an unbundled clone clearly does not read as integrated.
- **Loophole 2 — Optional cloud sync that the user can turn off:** A competitor offers optional cloud sync of room models and argues that because it is optional, the system “does not transmit” by default. **Fix:** Specify “in the default configuration of the application” or “in at least one configuration of the application available to the user without payment of additional fees.”
- **Loophole 3 — Separate the catalog from the visualization:** A competitor builds the visualization without an integrated catalog, then partners with retailers via deep-link. **Fix:** Make catalog integration a dependent claim.

6. Cross-Patent Consistency Notes

- **Door reference dimension inconsistency.** P1 §7B says door height ≈ 2.0 m. iOS code uses 2.1 m. Either change patent body to 2.1 m, or change patent body to “approximately 2.0 to 2.1 metres” to match either code path. This appears in P1 only; not a cross-patent issue, but worth catching.
- **“No LiDAR” framing varies.** P1 Claim 2 says “no LiDAR ... is used.” P3 Claim 4 says “without use of a LiDAR sensor.” P6 Claim 2 says “does not require a LiDAR sensor.” Only the P6 wording is consistent with the code. P1 and P3 should be conformed to the P6 wording (“does not require”) so that the iOS implementation that opportunistically uses LiDAR when present does not contradict the claims.
- **“Centimetre” claim inconsistency.** P1 Claim 16 (“centimetre-level dimensional accuracy”), P3 Claim 4 (“centimetre-level accuracy”), P4 Claim 12 (“within two centimetres”), P6 Step 4 description (consumer needs “within ± 2 cm”). Four different formulations of the accuracy promise across the bundle, none validated. Pick one validated formulation (or none — see proposed replacements above) and use it consistently.
- **Cross-platform sharing.** P6 Claim 11 says “shared platform-independent spatial computing layer.” P5 implies the same. The actual sharing is at the *model artefact* level, not the *code* level. Conform language across both patents.

- **Cloud / offline framing.** P1, P2, P5, P6 all assert “no cloud” / “no external server” / “no network connectivity.” Code uses Firebase Auth (account login) and Firebase initialization. None of these touch spatial data, but the patent language should narrowly say “no transmission of room photographic data, room three-dimensional model data, or measurement data” rather than “no cloud connectivity” — so that authentication, app updates, optional catalog refresh, etc., do not count as accidental contradictions.
- **Multi-frame vs single-image embodiment.** P1 entirely describes a multi-frame embodiment that does not exist in code; P6 implies the same multi-frame workflow. The single-image embodiment (which IS the code) is described nowhere. Add the single-image embodiment to P1 (as proposed above), and update P6’s “scanning session typically lasting 30 to 90 seconds” wording to allow for the single-image case (“a scanning session comprising one or more captured frames”).
- **Door / window opening output.** P3 Claim 7 promises door/window opening dimensions in the room summary. Not produced. Either remove from P3, or add a dependent claim describing it as an optional embodiment.

7. Filing-Day Triage Checklist

Strictly the items I would change before clicking submit, in priority order:

1. **DELETE Patent 2 Claim 5** (multi-frame segmentation consistency) **OR** be honest that it’s not implemented and rely on the 12-month non-provisional window to implement it. Same for Patent 4 Claims 7–8 (lighting estimation, shadow projection) and Patent 5 Claims 5 and 9 (sparse voxel, interruptible refinement). Filing primary claims for technology you do not have is the single biggest risk in this bundle.
2. **REPLACE Patent 1 Claim 2 / Patent 3 Claim 4 LiDAR exclusion** with the “does not require” wording from Patent 6 Claim 2. The current wording is contradicted by your own iOS code.
3. **REPLACE Patent 5 Claim 1** with the workload-routing-with-stability-conditioning version. The current “assign neural inference to NPU” claim is contradicted by your own `computeUnits = .cpuOnly` in production.
4. **ADD Patent 1 Claim 1A** (single-image neural Gaussian-splat embodiment). This is what your code actually does and right now nothing in any patent describes it.
5. **DELETE Patent 6 Claim 5** (commerce integration). Not built.
6. **DROP “centimetre” as a hard claim across all patents.** Move the centimetre-level statement to the SUMMARY as an aspirational target. Replace the underlying claim with the cv-gating uncertainty mechanism that *is* implemented.
7. **ADD the GAP-1 through GAP-10 prose** (§3 above) under the appropriate patent’s DETAILED DESCRIPTION before filing. Each one is something the code actually does that no patent currently describes. If you don’t add them, you cannot claim them in the non-provisional twelve months from now.
8. **CONFORM cross-patent wording** for “LiDAR,” “cloud,” “centimetre,” and “shared platform-independent layer” per §6.

Everything else is improvement; the eight items above are filing-blocking from a technical-accuracy standpoint.

End of audit.

This audit reflects the state of the codebase at /Users/al/Documents/tries01/Furnit/ as of the date below and the patent bundle as provided. It is technical analysis only and is not legal advice. A registered patent attorney should review any change to claim scope before filing.